In-Memory Computation with Apache Spark: Iris Dataset

Assignment No. 3

1. Environment

Item	Value
Apache Spark	4.3.0
Master	local[*]
Java	22.0.2
Python	3.12.8
Default parallelism	16
Dataset	UCI Iris, 150 rows, 4 features, 3 classes

All code in this report was executed locally on a Windows 11 machine. The Spark context runs in local mode, with all executors inside one JVM. Local mode shows how Spark behaves, but it does not measure cluster performance.

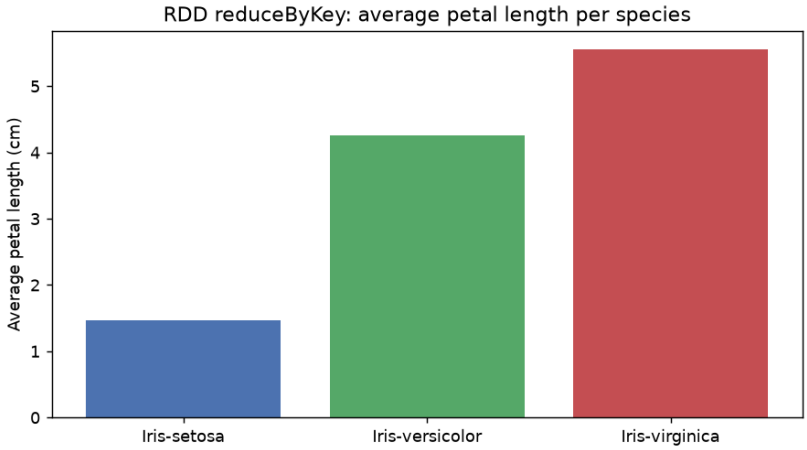
2. Spark context and RDDs

`SparkSession` wraps the `SparkContext`. The file is read with `textFile`, which creates an RDD with 2 partitions. Nothing is read until the first action runs. The pipeline has 150 records. Transformations (map, filter, mapValues, flatMap) build the plan; actions (count, first, collect, countByKey, reduce) run it.

Operation	Type	Result
filter (drop header)	transformation	lazy
map (split fields)	transformation	lazy
map (species, features)	transformation	lazy
count()	action	150
first()	action	Iris-setosa, 5.1, 3.5, 1.4, 0.2
reduceByKey (average petal length)	transformation + action	see table below
countByKey()	action	{'Iris-setosa': 50, 'Iris-versicolor': 50, 'Iris-virginica': 50}
flatMap + reduceByKey + sortByKey	transformations + action	per-feature maximum
reduce (sum of sepal length)	action	876.5

Species	Average petal length (cm)
Iris-setosa	1.464
Iris-versicolor	4.260
Iris-virginica	5.552

Max per feature: sepal_length 7.9, sepal_width 4.4, petal_length 6.9, petal_width 2.5. groupByKey gives the same averages as reduceByKey: **True**. reduceByKey combines values inside each partition before the shuffle, so it moves less data than groupByKey.


RDD reduceByKey: average petal length per species
Average petal length (cm)
0
1
2
3
4
5
Iris-setosa
Iris-versicolor
Iris-virginica

3. DataFrames and Spark SQL

Schema: `struct`. The file is read with an explicit schema, so types are not inferred. The filter keeps flowers with sepal length of at least 5.0 cm (128 rows). A derived column (petal ratio) is added without a shuffle.

Species	n	avg sepal length	avg sepal width	avg petal length	avg petal width	stddev petal length
Iris-setosa	50	5.006	3.418	1.464	0.244	0.174
Iris-versicolor	50	5.936	2.77	4.26	1.326	0.47
Iris-virginica	50	6.588	2.974	5.552	2.026	0.552

Top 3 flowers by sepal length in each species (window function row_number):

Species	Rank	Sepal length	Petal length
Iris-setosa	1	5.8	1.2
Iris-setosa	2	5.7	1.5
Iris-setosa	3	5.7	1.7
Iris-versicolor	1	7.0	4.7
Iris-versicolor	2	6.9	4.9
Iris-versicolor	3	6.8	4.8
Iris-virginica	1	7.9	6.4
Iris-virginica	2	7.7	6.7
Iris-virginica	3	7.7	6.9

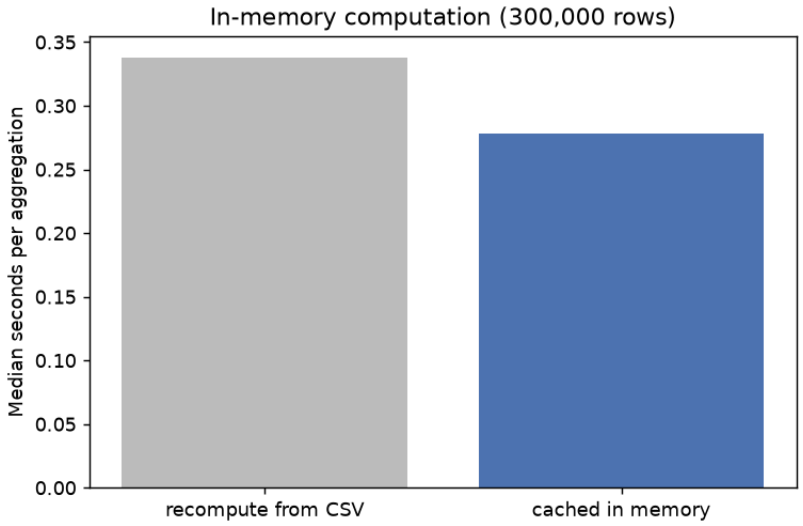
The join with a small lookup table uses a broadcast hint, so the lookup is copied to each executor instead of shuffling the large side. The same aggregation in Spark SQL is planned by the same optimizer.

4. In-memory computation

Iris has only 150 rows, too small to show a caching effect. The benchmark replicates the data by a cross join with 2,000 copies (300,000 rows). It runs the same aggregation three times (median time) in two ways: recomputing from the CSV each time, and reading from a cache that was materialised once with `persist(MEMORY_ONLY)`.

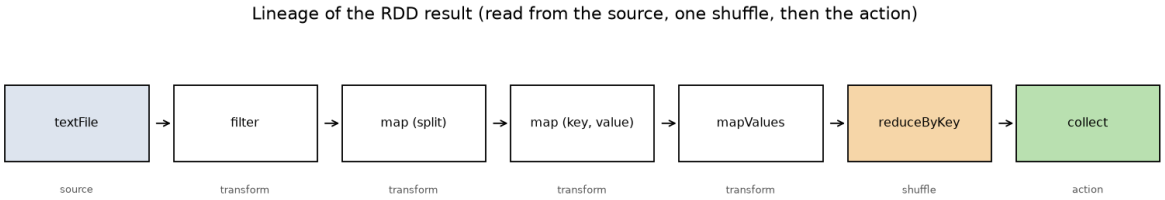
Mode	Median seconds	Storage level
Recompute from CSV each time	0.338	none
Cached in memory	0.278	Memory Serialized 1x Replicated

Speedup 1.21x in this run. Timings are from a single local machine, so they vary between runs. With the cache, each query skips re-reading the CSV and rebuilding the cross join. The cache is only useful when the same data is reused, as in this benchmark.



5. Data lineage

Lineage is the chain of transformations that produces each result. Spark keeps it as an RDD dependency graph or a logical plan, and uses it to recompute lost partitions. The RDD debug strings below show each stage. Indentation marks a shuffle boundary. The DataFrame plans show the parsed, analysed, optimised and physical forms.



Full lineage output (first 40 lines of each section, from lineage.txt):

```
===== RDD: reduceByKey_lineage =====
(2) PythonRDD[9] at collect at C:\Users\308638\Script Automation\AMITY-UNIVERSITY\assignment3\iris_spark
 |  MapPartitionsRDD[8] at mapPartitions at PythonRDD.scala:197 []
 |  ShuffledRDD[7] at partitionBy at <unknown>:0 []
 +-(2) PairwiseRDD[6] at reduceByKey at C:\Users\308638\Script Automation\AMITY-UNIVERSITY\assignment3\i
    |  PythonRDD[5] at reduceByKey at C:\Users\308638\Script Automation\AMITY-UNIVERSITY\assignment3\iri
    |  data\iris.csv MapPartitionsRDD[1] at textFile at <unknown>:0 []
    |  data\iris.csv HadoopRDD[0] at textFile at <unknown>:0 []

===== RDD: groupByKey_lineage =====
(2) PythonRDD[15] at collect at C:\Users\308638\Script Automation\AMITY-UNIVERSITY\assignment3\iris_spar
 |  MapPartitionsRDD[14] at mapPartitions at PythonRDD.scala:197 []
 |  ShuffledRDD[13] at partitionBy at <unknown>:0 []
 +-(2) PairwiseRDD[12] at groupByKey at C:\Users\308638\Script Automation\AMITY-UNIVERSITY\assignment3\i
    |  PythonRDD[11] at groupByKey at C:\Users\308638\Script Automation\AMITY-UNIVERSITY\assignment3\iri
    |  data\iris.csv MapPartitionsRDD[1] at textFile at <unknown>:0 []
    |  data\iris.csv HadoopRDD[0] at textFile at <unknown>:0 []

===== DataFrame: dataframe_aggregation_plan =====
== Parsed Logical Plan ==
'Sort ['species ASC NULLS FIRST], true
+- 'Aggregate ['species], ['species, 'ROUND('AVG('petal_length), 3) AS avg_petal_length#139]
   +- 'UnresolvedRelation [iris], [], false

== Analyzed Logical Plan ==
species: string, avg_petal_length: double
Sort [species#4 ASC NULLS FIRST], true
+- Aggregate [species#4], [species#4, round(avg(petal_length#2), 3) AS avg_petal_length#139]
   +- SubqueryAlias iris
      +- View (`iris`, [sepal_length#0, sepal_width#1, petal_length#2, petal_width#3, species#4])
         +- Filter NOT (species#4 = class)
            +- Relation [sepal_length#0,sepal_width#1,petal_length#2,petal_width#3,species#4] csv

== Optimized Logical Plan ==
Sort [species#4 ASC NULLS FIRST], true
+- Aggregate [species#4], [species#4, round(avg(petal_length#2), 3) AS avg_petal_length#139]
   +- Project [petal_length#2, species#4]
      +- Filter (isnotnull(species#4) AND NOT (species#4 = class))
         +- Relation [sepal_length#0,sepal_width#1,petal_length#2,petal_width#3,species#4] csv
```

6. Spark UI

The Spark UI was reachable at http://10.11.87.32:4040 during the run. Its REST API was exported to `output/spark_ui/`: jobs (46), stages (73), storage/rdd (1), executors (1), sql (17). The storage export shows the cached RDD while it is held in memory. The SQL export lists each query with its plan.

Screenshots: the Spark UI pages (Jobs, Stages, Storage, SQL) are shown when the script runs with `--pause`. They are not included in this report because they must be taken from a browser while Spark is running.

7. Limitations

- Local mode runs on one machine. The results show Spark's behaviour, not cluster performance.

- Iris is small. The caching benchmark replicates it to make the effect measurable.

- Timings come from one run and vary between runs.